

EE 446: TinyML

Assignment 2: Programming Results Writeup

Student Information

Full Name: Luke Valerio

Student ID: 1901711

Repository: https://github.com/LukeVal27/tiny_ML

Note on this document. Assignment 2 contains no questions marked [Dis] — all five problems are [Pro] and sum to 100 points. The handout still requires a PDF named *FirstName_LastName_EE446_HW2_Dis.pdf*, and problems 3(c), 4(b), 5(c) and 5(d) each ask for results to be reported in "your .pdf writeup file using screenshots." This document therefore collects those four sets of results. All numbers come from *Luke_Valerio_EE446_HW2_Pro.ipynb*, executed end to end with `random_state=42` so the train/test split is reproducible.

Question 3(c) — float32 base_model on the test set

The DNN is a four-layer fully connected network ($38 \rightarrow 64 \rightarrow 32 \rightarrow 16 \rightarrow 1$) with ReLU hidden activations and a single sigmoid output neuron, trained for 20 epochs with Adam and binary cross-entropy. It was kept deliberately small so that it fits the 32 KB tensor arena allocated in `network_data.ino`.

Q3(c) FLOAT32 base_model - test set performance
Accuracy: 0.9985

	precision	recall	f1-score	support
attack (0)	0.9984	0.9985	0.9984	11726
normal (1)	0.9987	0.9986	0.9986	13469
accuracy			0.9985	25195
macro avg	0.9985	0.9985	0.9985	25195
weighted avg	0.9985	0.9985	0.9985	25195

Confusion matrix (rows = actual, cols = predicted):
[[11708 18]
[19 13450]]

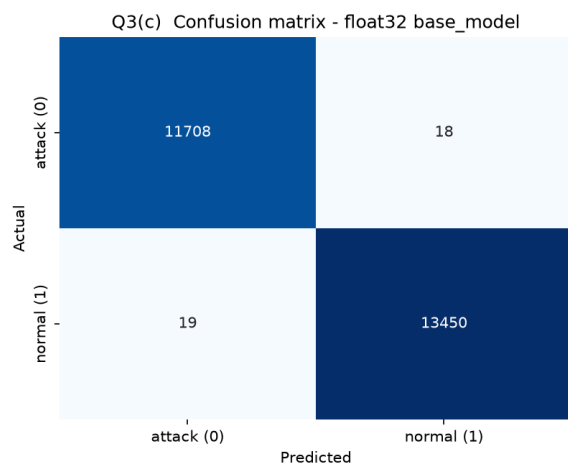


Figure 1. Confusion matrix, float32 base_model, 25,195 test samples.

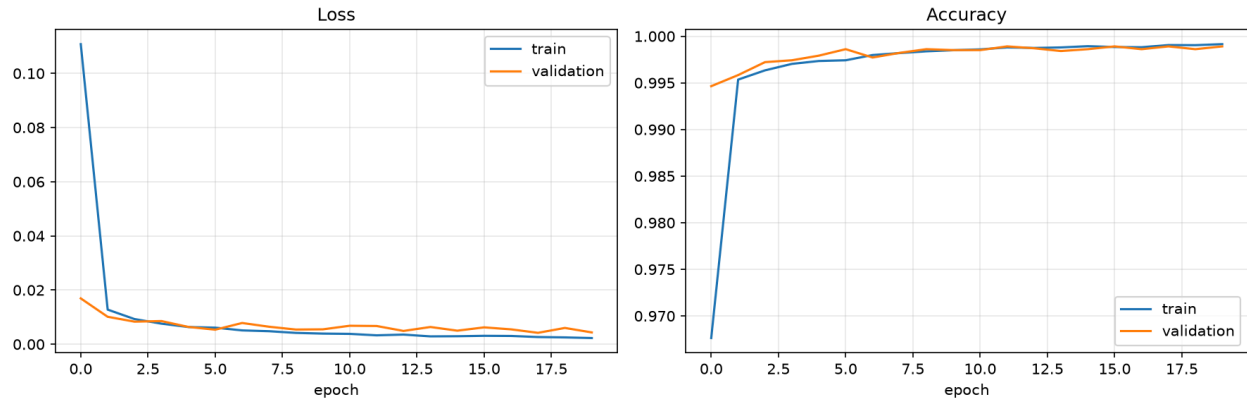


Figure 2. Training and validation loss/accuracy over 20 epochs.

Question 4(b) — INT8 tfliite_quant_model on the test set

Full-integer post-training quantization was applied with a 500-sample representative dataset drawn from X_train to calibrate activation ranges. The converter was restricted to TFLITE_BUILTINS_INT8 operators with inference_input_type and inference_output_type both set to tf.int8, so no float operator survives in the deployed graph. Each test sample is quantized with the input tensor's own scale and zero point before inference, and the INT8 result is dequantized with the output tensor's scale and zero point.

Q4(b) INT8 tfliite_quant_model - test set performance

input scale = 0.0895969868 zero_point = -52
output scale = 0.0039062500 zero_point = -128

Accuracy: 0.9984

	precision	recall	f1-score	support
attack (0)	0.9979	0.9987	0.9983	11726
normal (1)	0.9989	0.9981	0.9985	13469
accuracy			0.9984	25195
macro avg	0.9984	0.9984	0.9984	25195
weighted avg	0.9984	0.9984	0.9984	25195

Confusion matrix (rows = actual, cols = predicted):

```
[[11711  15]
 [ 25 13444]]
```

float32 accuracy : 0.9985
INT8 accuracy : 0.9984
change : -0.0001

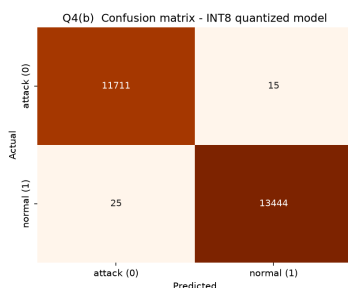


Figure 3. Confusion matrix, INT8 quantized model, 25,195 test samples.

Float32 vs INT8 comparison

Model	Size	Test accuracy
float32 Keras (original_model.h5)	100.01 KB	0.9985
INT8 TFLite (quantized_model.tflite)	8.24 KB	0.9984
Change	12.1× smaller	−0.0001

Full-integer quantization cost one hundredth of a percentage point of accuracy — three additional misclassifications out of 25,195 — while shrinking the model by more than an order of magnitude. That trade is what makes the model deployable on the Nano 33 BLE.

Question 5(c) — Serial Monitor output, 5 samples

Sketch: network_data/network_data.ino, carrying rows 0–4 of X_test. The INT8 output tensor is dequantized as $(\text{value} - \text{zero_point}) \times \text{scale}$ and thresholded at 0.5. Quantization parameters baked into the model: input scale = 0.0895969868, zero point = −52; output scale = 0.0039062500, zero point = −128.

Output	Serial Monitor	×
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem101')		
Sample #1	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #2	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #3	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #4	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #0	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #1	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #2	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #3	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #4	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #0	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #1	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #2	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #3	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #4	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #0	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)
Sample #1	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #2	Predicted Class: 1	Actual Class: 1 (sigmoid = 0.9961)
Sample #3	Predicted Class: 0	Actual Class: 0 (sigmoid = 0.0000)

Figure 4. Serial Monitor output on the Arduino Nano 33 BLE for the five test samples. `loop()` repeats indefinitely, so several identical cycles of Sample #0–#4 are visible. All five predictions match the actual labels, and they reproduce the desktop TFLite interpreter exactly.

Question 5(d) — Serial Monitor output, 10 samples

Sketch: network_data_10/network_data_10.ino, carrying rows 5–14 of X_test — the ten samples that follow the five used in part (c), as required.

```
Output  Serial Monitor ×
Message (Enter to send message to 'Arduino Nano 33 BLE' on '/dev/cu.usbmodem101')

Sample #0 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #1 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #2 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #3 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #4 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #5 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #6 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #7 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #8 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #9 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #0 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #1 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #2 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #3 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #4 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #5 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #6 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #7 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
Sample #8 | Predicted Class: 0 | Actual Class: 0 | (sigmoid = 0.0000)
Sample #9 | Predicted Class: 1 | Actual Class: 1 | (sigmoid = 0.9961)
```

Figure 5. Serial Monitor output on the Arduino Nano 33 BLE for the ten test samples (two complete cycles shown). All ten predictions match the actual labels: six attack and four normal.

Every sigmoid value reads either 0.0000 or 0.9961, never anything between. This is the INT8 output tensor saturating rather than a defect. With output scale 1/256 and zero point -128, an INT8 value of -128 dequantizes to exactly 0.0 and the maximum value of +127 dequantizes to $255/256 = 0.99609375$. A model this confident pins to both rails, so the quantized sigmoid can only report the endpoints.

Expected on-device predictions

Produced by the desktop TFLite interpreter on the same quantized model, for cross-checking the Serial Monitor output. All 15 samples are classified correctly.

```
Expected predictions from the desktop INT8 interpreter
input  scale=0.0895969868 zero_point=-52
output scale=0.0039062500 zero_point=-128

Q5(c) samples 0-4
Sample 0: prob=0.0000 predicted=0 actual=0
Sample 1: prob=0.9961 predicted=1 actual=1
Sample 2: prob=0.9961 predicted=1 actual=1
Sample 3: prob=0.0000 predicted=0 actual=0
Sample 4: prob=0.0000 predicted=0 actual=0

Q5(d) samples 5-14
Sample 5: prob=0.0000 predicted=0 actual=0
Sample 6: prob=0.9961 predicted=1 actual=1
```

```
Sample 7: prob=0.0000 predicted=0 actual=0
Sample 8: prob=0.9961 predicted=1 actual=1
Sample 9: prob=0.0000 predicted=0 actual=0
Sample 10: prob=0.0000 predicted=0 actual=0
Sample 11: prob=0.0000 predicted=0 actual=0
Sample 12: prob=0.9961 predicted=1 actual=1
Sample 13: prob=0.0000 predicted=0 actual=0
Sample 14: prob=0.9961 predicted=1 actual=1
```